

Assignment – 11.5

Name: P.Sanjana

Batch: 21

Ht.no: 2303A51435

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

```
class Stack:
```

```
    pass
```

Expected Output:

- A functional stack implementation with all required methods and docstrings.

Prompt Used to AI

“Generate a Python Stack class with push, pop, peek, and is_empty methods with comments and docstrings.”

```
20-02-2026.py > ...
1  #Generate a Python Stack class with push, pop, peek, and is_empty methods with comments and doc
2  class Stack:
3      """A simple implementation of a stack data structure."""
4
5      def __init__(self):
6          """Initialize an empty stack."""
7          self.items = []
8
9      def push(self, item):
10         """Add an item to the top of the stack."""
11         self.items.append(item)
12
13     def pop(self):
14         """Remove and return the item at the top of the stack. Raises an error if the stack is e
15         if self.is_empty():
16             raise IndexError("Pop from an empty stack")
17         return self.items.pop()
18
19     def peek(self):
20         """Return the item at the top of the stack without removing it. Raises an error if the s
21         if self.is_empty():
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
eDrive\Desktop\AI\20-02-2026.py'
2
2
False
1
True
PS C:\Users\dell\OneDrive\Desktop\AI>
```

◆ Observation

The Stack data structure works in LIFO order. AI-generated code improved clarity using docstrings and error handling.

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
```

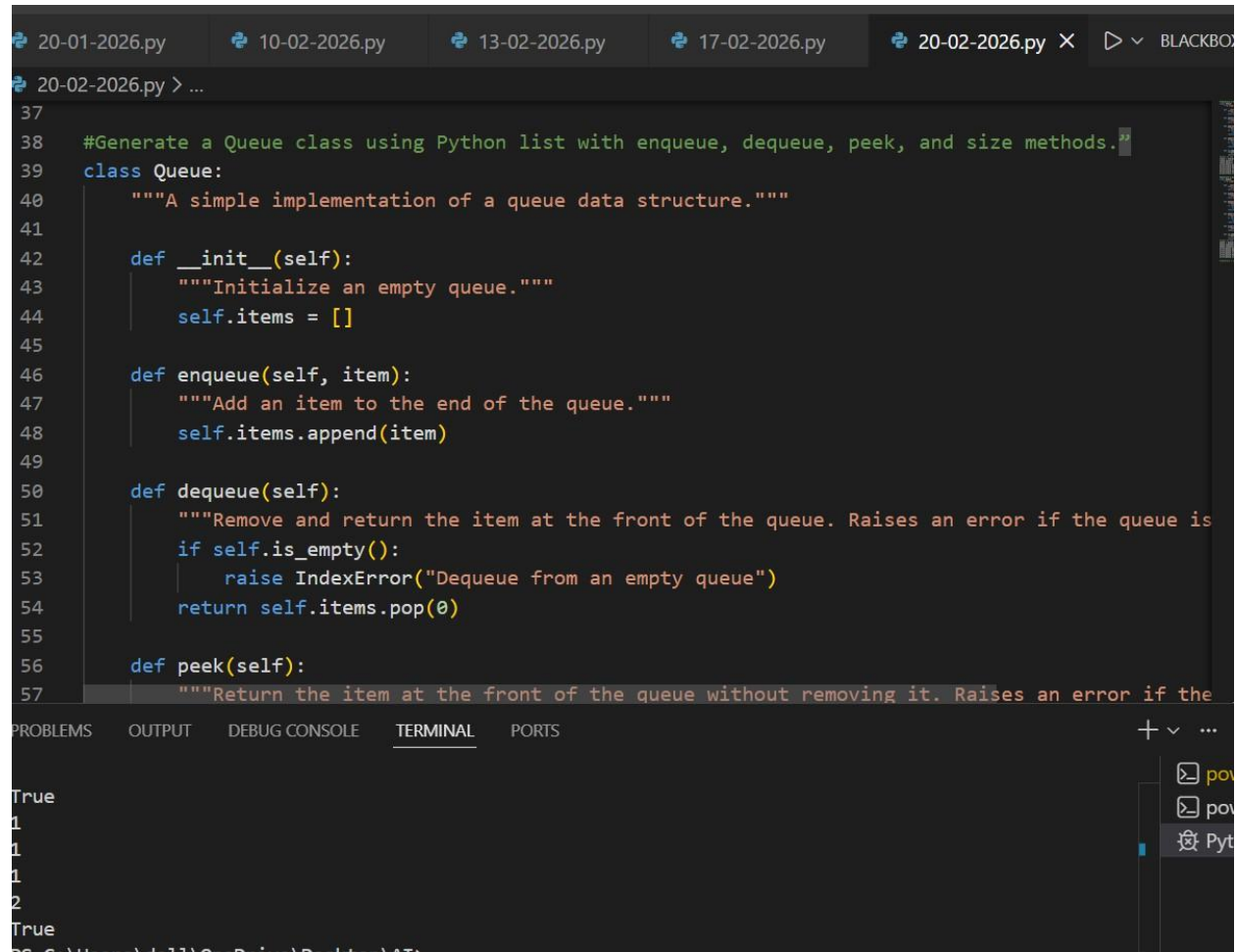
```
pass
```

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size Methods

Prompt

“Generate a Queue class using Python list with enqueue, dequeue, peek, and size methods.”

A screenshot of a Python IDE window with multiple tabs. The active tab is '20-02-2026.py'. The code defines a 'Queue' class using a Python list. The class has methods for enqueue, dequeue, and peek. The dequeue method includes a check for an empty queue to raise an IndexError. The peek method is partially visible at the bottom of the code block.

```
37
38 #Generate a Queue class using Python list with enqueue, dequeue, peek, and size methods.
39 class Queue:
40     """A simple implementation of a queue data structure."""
41
42     def __init__(self):
43         """Initialize an empty queue."""
44         self.items = []
45
46     def enqueue(self, item):
47         """Add an item to the end of the queue."""
48         self.items.append(item)
49
50     def dequeue(self):
51         """Remove and return the item at the front of the queue. Raises an error if the queue is
52         if self.is_empty():
53             raise IndexError("Dequeue from an empty queue")
54         return self.items.pop(0)
55
56     def peek(self):
57         """Return the item at the front of the queue without removing it. Raises an error if the
```

◆ Observation

Queue follows FIFO principle. AI helped in designing efficient queue operations.

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

```
class Node:
    pass
```

```
class LinkedList:
```

```
pass
```

Expected Output:

- A working linked list implementation with clear method documentation.

Prompt: “Generate a singly linked list with insert and display methods.”

```
20-02-2026.py > ...
78
79 #“Generate a singly linked list with insert and display methods.”
80 class Node:
81     """A node in a singly linked list."""
82
83     def __init__(self, data):
84         """Initialize a node with data and a pointer to the next node."""
85         self.data = data
86         self.next = None
87 class SinglyLinkedList:
88     """A simple implementation of a singly linked list."""
89
90     def __init__(self):
91         """Initialize an empty linked list."""
92         self.head = None
93
94     def insert(self, data):
95         """Insert a new node with the given data at the end of the list."""
96         new_node = Node(data)
97         if not self.head:
98             self.head = new_node
99             return
100         last_node = self.head
101         while last_node.next:
102             last_node = last_node.next
103         last_node.next = new_node
```

◆ Observation

Linked list stores nodes dynamically. AI-generated structure simplified pointer handling.

Task Description #4 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

```
class HashTable:
```

pass

Expected Output:

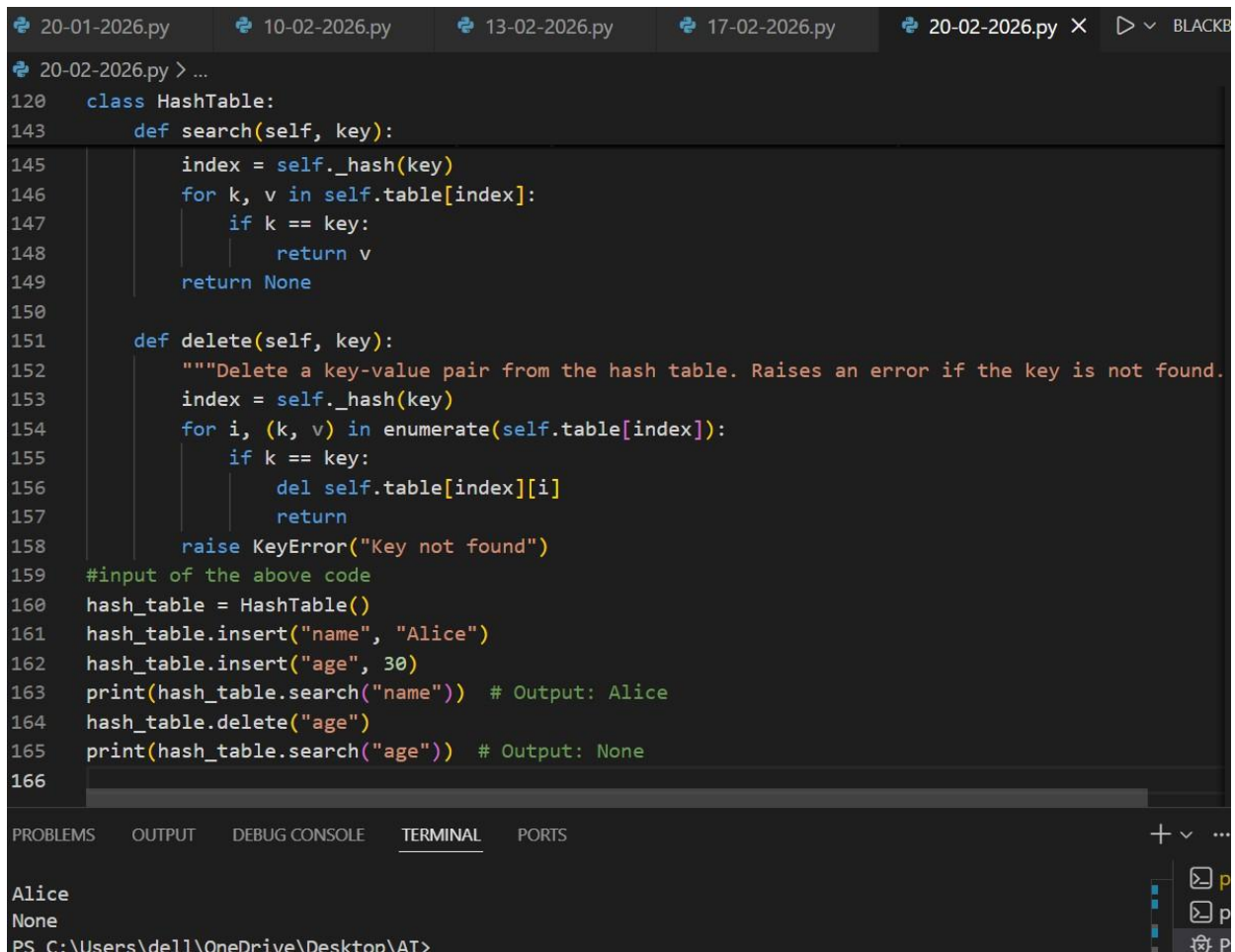
- Collision handling using chaining, with well-commented methods.

Prompt

“Generate a Hash Table class with insert, search, delete using chaining.”

◆ Observation

Hash table enables fast $O(1)$ lookup. AI suggested chaining to handle collisions.



```
20-01-2026.py 10-02-2026.py 13-02-2026.py 17-02-2026.py 20-02-2026.py X BLACKB
20-02-2026.py > ...
120 class HashTable:
143     def search(self, key):
145         index = self._hash(key)
146         for k, v in self.table[index]:
147             if k == key:
148                 return v
149         return None
150
151     def delete(self, key):
152         """Delete a key-value pair from the hash table. Raises an error if the key is not found.
153         index = self._hash(key)
154         for i, (k, v) in enumerate(self.table[index]):
155             if k == key:
156                 del self.table[index][i]
157                 return
158         raise KeyError("Key not found")
159
160 #input of the above code
160 hash_table = HashTable()
161 hash_table.insert("name", "Alice")
162 hash_table.insert("age", 30)
163 print(hash_table.search("name")) # Output: Alice
164 hash_table.delete("age")
165 print(hash_table.search("age")) # Output: None
166
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Alice
None
PS C:\Users\dell\OneDrive\Desktop\AT>

Task Description #5 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

```
class Graph:
```

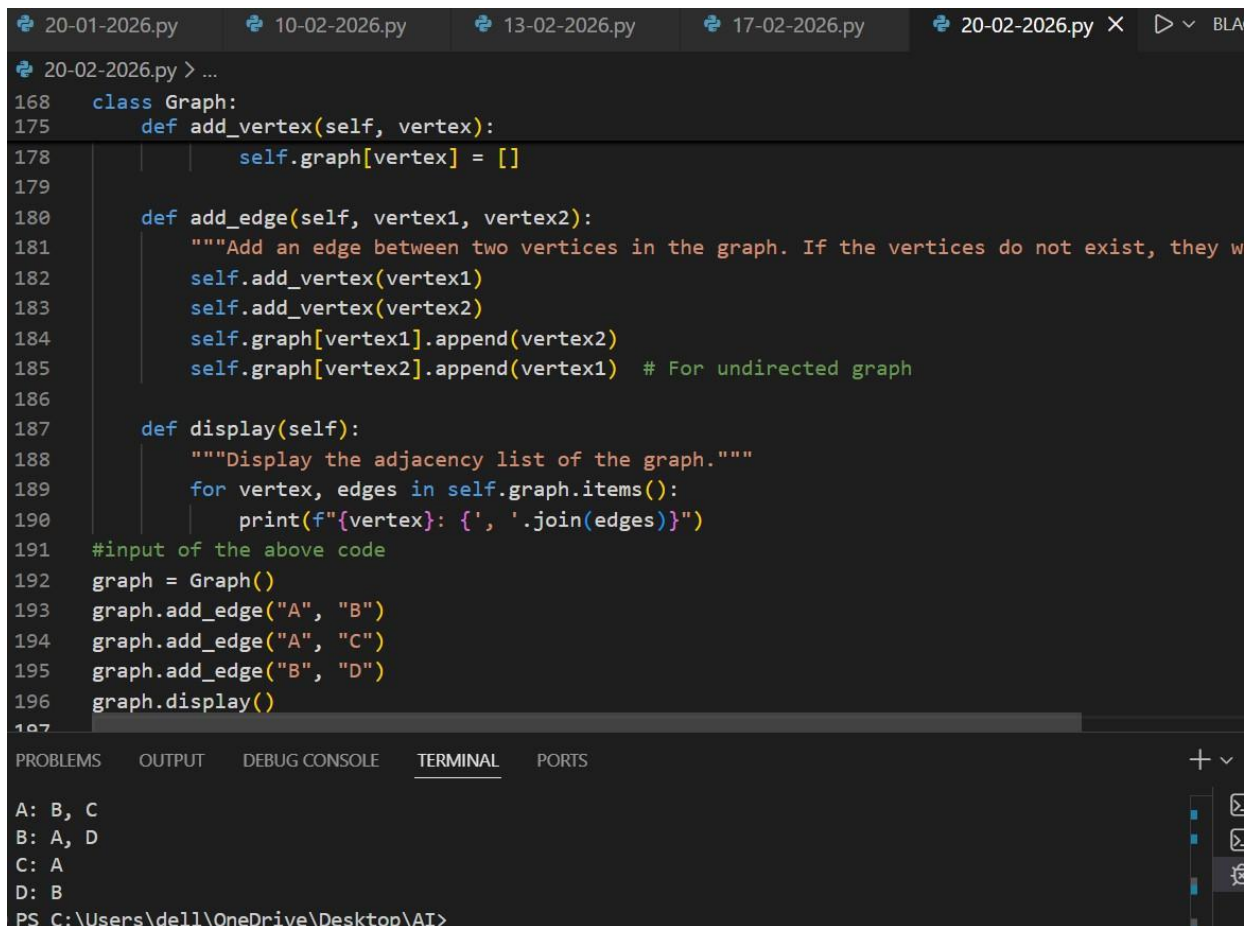
pass

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

Prompt

“Generate a Graph class using adjacency list with add_vertex, add_edge, display methods.”



```
20-01-2026.py 10-02-2026.py 13-02-2026.py 17-02-2026.py 20-02-2026.py X BLA
20-02-2026.py > ...
168 class Graph:
175     def add_vertex(self, vertex):
178         self.graph[vertex] = []
179
180     def add_edge(self, vertex1, vertex2):
181         """Add an edge between two vertices in the graph. If the vertices do not exist, they w
182         self.add_vertex(vertex1)
183         self.add_vertex(vertex2)
184         self.graph[vertex1].append(vertex2)
185         self.graph[vertex2].append(vertex1) # For undirected graph
186
187     def display(self):
188         """Display the adjacency list of the graph."""
189         for vertex, edges in self.graph.items():
190             print(f"{vertex}: {' '.join(edges)}")
191 #input of the above code
192 graph = Graph()
193 graph.add_edge("A", "B")
194 graph.add_edge("A", "C")
195 graph.add_edge("B", "D")
196 graph.display()
197
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
A: B, C
B: A, D
C: A
D: B
PS C:\Users\dell\OneDrive\Desktop\AI>
```

◆ Observation

Graph adjacency list efficiently stores network connections. AI helped in structuring graph logic.

Task Description #6: Smart Hospital Management System – Data

Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Feature	Data Structure Justification	
Patient Check-In	Queue	FIFO order for patient arrival
Emergency Handling	Priority Queue	Critical patients treated first
Medical Records	Hash Table	Fast ID-based retrieval
Appointment Scheduling	BST	Sorted by time
Room Navigation	Graph	Represents connected rooms

Task Description #7: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.
2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.
3. Vehicle Registration Lookup – Instant access to vehicle details.
4. Road Network Mapping – Roads and intersections connected logically.
5. Parking Slot Availability – Track available and occupied slots.

Student Task

- For each feature, select the most appropriate data structure from the list below:

- o Stack
- o Queue
- o Priority Queue
- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Feature	Data Structure	Justification
Traffic Signal Queue	Queue	Vehicles wait in order
Emergency Priority	Priority Queue	Ambulances first
Vehicle Lookup	Hash Table	Instant access
Road Network	Graph	Intersections and roads
Parking Slots	Deque	Add/remove from both ends

Task Description #8: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Feature	Data Structure	Justification
Shopping Cart	Linked List	Dynamic add/remove
Order Processing	Queue	FIFO order
Top Products	Priority Queue	Ranked by sales
Product Search	Hash Table	Fast lookup
Delivery Routes	Graph	Location connections